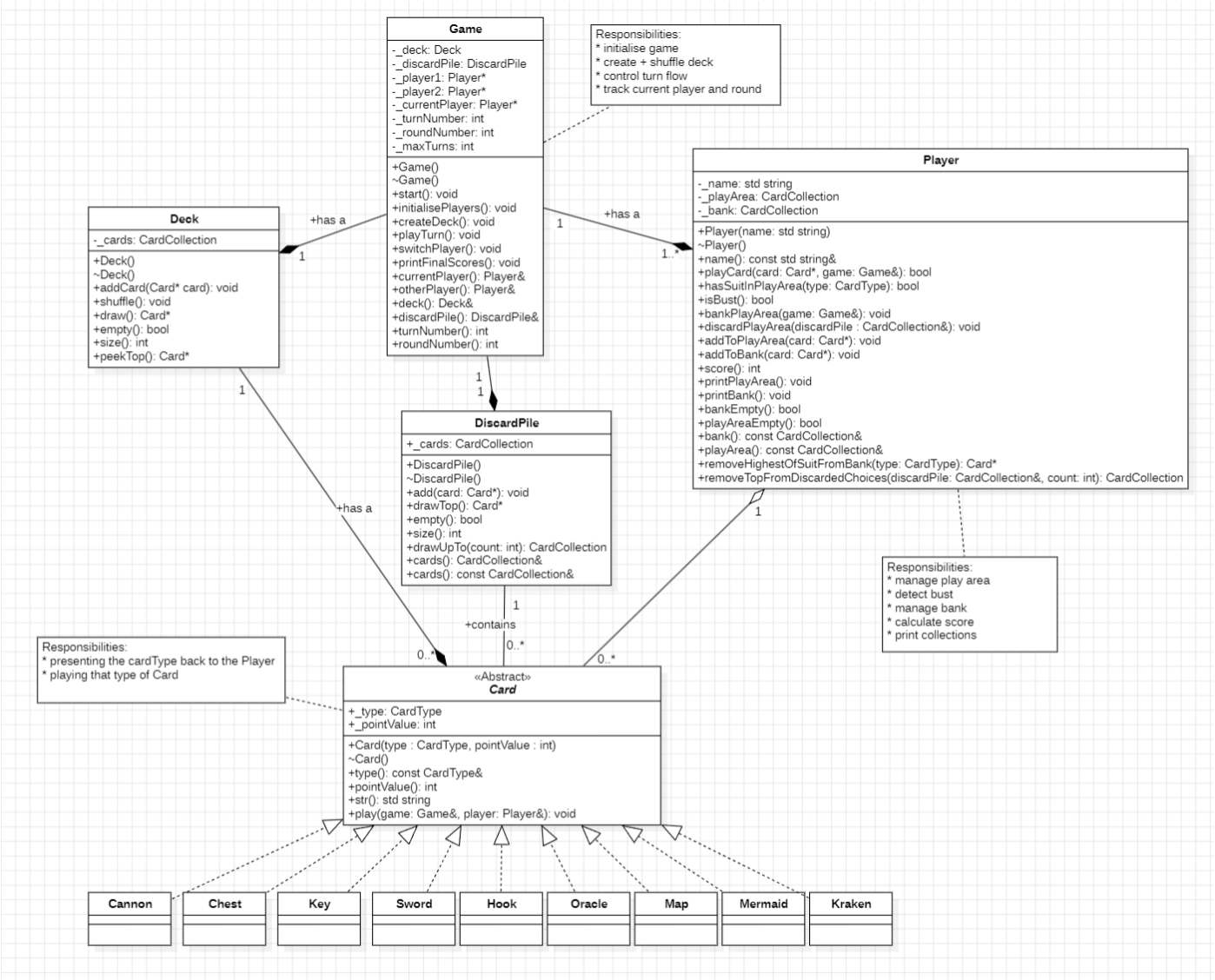
System Design Document

Name: Simon Johnson

Email Address: simon.johnson@student.adelaide.edu.au

Student ID: a2708173

UML Class Diagram



Design Documentation

- The primary design pattern used in this system design is the Strategy Pattern, where the abstract Card class defines a common interface for Card behaviour, and each derived Card class provides its own implementation of play(). This allows card abilities/effects to vary independently while the rest of the game interacts with all Cards through the same polymorphic interface.
- The Strategy Pattern was selected to encapsulate different Card behaviours into separate classes, allowing them to be used polymorphically through a common interface. This approach improves maintainability, supports extensibility, and encapsulates complex conditional logic from the main game flow.
- A Game class was used to centralise overall game control, including setup, turn flow, round tracking, Player switching, and access to shared resources such as the Deck and Discard Pile
- The Player class is responsible for managing Player-specific state such as including the play area, bank, score calculation, and bust detection so that these behaviours remain separate from the overall game flow.
- A base abstract Card class was used to represent shared Card data and behaviour such as CardType, pointValue, and common interfaces such as str() and play().
- Inheritance was used for the individual Card types (Cannon, Chest, Key, Sword, Hook, Oracle, Map, Mermaid, Kraken) so that each Card could implement its own special ability while still being treated polymorphically as a Card.
- The willAddToBank(Game&, Player&) method was included in the Card class to support special abilities that occur when a Card is banked rather than when the Card is first played, such as the Chest + Key interaction.
- The Deck class was separated from Game to encapsulate Card storage and Deck specific behaviour such as adding Cards, shuffling, drawing, peeking and checking if the Deck is empty.
- The DiscardPile class was implemented separately to manage discarded Cards independently from the Deck, allowing Card abilities such as Map and Chest to interact with discarded Cards without overcomplicating the Game and Player classes.
- CardCollection was defined as a collection of Card* so that different derived CardTypes could be stored together in the same container and handled through polymorphism.
- Game stores Players as pointers so that the currentPlayer can be switched efficiently by changing which Player object _currentPlayer refers to, rather than copying whol Player objects.
- The Player::playCard() method was created to centralise the process of adding a card to the playArea, resolving its ability effect, and checking for bust conditions, keeping this repeated logic in one place, supporting modularity.
- The Player::bankPlayArea() method was used to encapsulate all logic related to moving Cards from the playArea to the bank, including triggering any relevant banking effects.
- The Player::discardPlayArea() method was included to handle busts by consistently moving all Cards in the playArea into the DiscardPile.
- Accessor methods such as deck(), discardPile(), currentPlayer(), and otherPlayer() were included in Game so that Card abilities/effects can interact with shared Game state without exposing the internal data members directly (encapsulation).
- Input validation was added to Player choice flows such as selecting Cards or suits to prevent invalid input from crashing the program or placing the Game into an inconsistent state.
- The Oracle Card resolves the drawn Card directly within OracleCard::play() rather than calling the higher-level playCard() method again to avoid re-entering the main turn flow and causing unintended extra actions.
- The Map Card was designed to draw a temporary set of Cards from the DiscardPile, allow the Player to choose exactly one Card to play, and return the unchosen Cards, ensuring that only one selected Card ability/effect is resolved.
- Responsibilities were distributed across classes to support separation of concerns, encapsulation, and modularity, making the system design easier to read, test, debug, and extend.